

SQL BOOK 2

=====

SQL BOOK 2**Exercise-2**

=====

From this exercise onwards all solutions of SQL are given after the last chapter.

Create the following tables with necessary constraints.

WORKER (WORKER_ID, NAME, WAGE_PER_HOUR, SPECIALISED_IN, MANAGER_ID)

JOB (JOB_ID, TYPE_OF_JOB, STATUS)

JOB_ASSIGNED (WORKER_ID, JOB_ID, STARTING_DATE, NUMBER_OF_DAYS)

1. SQL command to create the WORKER table:

```
CREATE TABLE WORKER
(
  WORKER_ID    NUMBER(5) PRIMARY KEY,
  NAME         VARCHAR2(50) NOT NULL,
  WAGE_PER_HOUR NUMBER(8,2) CHECK (WAGE_PER_HOUR > 0),
  SPECIALISED_IN VARCHAR2(50),
  MANAGER_ID   NUMBER(5)
);
```

2. SQL command to create the JOB table:

```
CREATE TABLE JOB
(
  JOB_ID    NUMBER(5) PRIMARY KEY,
  TYPE_OF_JOB VARCHAR2(50) NOT NULL,
  STATUS    VARCHAR2(20)
);
```

3. SQL command to create the JOB_ASSIGNED table:

```
CREATE TABLE JOB_ASSIGNED
(
  WORKER_ID    NUMBER(5) REFERENCES WORKER(WORKER_ID),
  JOB_ID       NUMBER(5) REFERENCES JOB(JOB_ID),
  STARTING_DATE DATE,
  NUMBER_OF_DAYS NUMBER(5) CHECK (NUMBER_OF_DAYS > 0)
);
```

4. Display the date on which each worker is going to end his presently assigned job.

SQL BOOK 2

```
SQL> SELECT
2     W.WORKER_ID,
3     W.NAME,
4     JA.STARTING_DATE,
5     JA.NUMBER_OF_DAYS,
6     JA.STARTING_DATE + JA.NUMBER_OF_DAYS AS ENDING_DATE
7 FROM WORKER W
8 JOIN JOB_ASSIGNED JA
9     ON W.WORKER_ID = JA.WORKER_ID;
```

WORKER_ID	NAME	STARTING_	NUMBER_OF_DAYS	ENDING_DA
104	Robert	15-DEC-26	20	04-JAN-27
105	Smith	01-OCT-04	30	31-OCT-04
102	John	10-DEC-26	15	25-DEC-26
103	David	15-MAY-00	25	09-JUN-00
106	Peter	12-AUG-26	10	22-AUG-26
101	Cacophonix	20-AUG-26	12	01-SEP-26

6 rows selected.

5. Display how many days remain for each worker to finish his job.

```
SQL> SELECT
2     W.WORKER_ID,
3     W.NAME,
4     JA.STARTING_DATE,
5     JA.NUMBER_OF_DAYS,
6     GREATEST(
7         0,
8         CEIL(JA.STARTING_DATE + JA.NUMBER_OF_DAYS - SYSDATE)
9     ) AS DAYS_REMAINING
10 FROM WORKER W
11 JOIN JOB_ASSIGNED JA
12     ON W.WORKER_ID = JA.WORKER_ID;
```

WORKER_ID	NAME	STARTING_	NUMBER_OF_DAYS
104	Robert	15-DEC-26	20
105	Smith	01-OCT-04	30
102	John	10-DEC-26	15
103	David	15-MAY-00	25
106	Peter	12-AUG-26	10
101	Cacophonix	20-AUG-26	12

6 rows selected.

6. Display the STARTING_DATE in the following format - 'The fifth day of the month of October, 2004'.

```

SQL> SELECT
2      CASE TO_CHAR(STARTING_DATE, 'DD')
3          WHEN '01' THEN 'The first'
4          WHEN '02' THEN 'The second'
5          WHEN '03' THEN 'The third'
6          WHEN '04' THEN 'The fourth'
7          WHEN '05' THEN 'The fifth'
8          WHEN '06' THEN 'The sixth'
9          WHEN '07' THEN 'The seventh'
10         WHEN '08' THEN 'The eighth'
11         WHEN '09' THEN 'The ninth'
12         WHEN '10' THEN 'The tenth'
13         WHEN '11' THEN 'The eleventh'
14         WHEN '12' THEN 'The twelfth'
15         WHEN '13' THEN 'The thirteenth'
16         WHEN '14' THEN 'The fourteenth'
17         WHEN '15' THEN 'The fifteenth'
18         WHEN '16' THEN 'The sixteenth'
19         WHEN '17' THEN 'The seventeenth'
20         WHEN '18' THEN 'The eighteenth'
21         WHEN '19' THEN 'The nineteenth'
22         WHEN '20' THEN 'The twentieth'
23         WHEN '21' THEN 'The twenty-first'
24         WHEN '22' THEN 'The twenty-second'
25         WHEN '23' THEN 'The twenty-third'
26         WHEN '24' THEN 'The twenty-fourth'
27         WHEN '25' THEN 'The twenty-fifth'
28         WHEN '26' THEN 'The twenty-sixth'
29         WHEN '27' THEN 'The twenty-seventh'
30         WHEN '28' THEN 'The twenty-eighth'
31         WHEN '29' THEN 'The twenty-ninth'
32         WHEN '30' THEN 'The thirtieth'
33         WHEN '31' THEN 'The thirty-first'
34     END
35     || ' day of the month of '
36     || TO_CHAR(STARTING_DATE, 'Month')
37     || ', '
38     || TO_CHAR(STARTING_DATE, 'YYYY') AS FORMATTED_DATE
39 FROM JOB_ASSIGNED;

```

FORMATTED_DATE

```

-----
The fifteenth day of the month of December , 2026
The first day of the month of October , 2004
The tenth day of the month of December , 2026
The fifteenth day of the month of May , 2000
The twelfth day of the month of August , 2026
The twentieth day of the month of August , 2026

```

6 rows selected.

7. Give the necessary access rights to user SUPERVISOR so that he can only assign jobs to different workers.

8. Change the status to 'Complete' for all those jobs, which started in year 2000.

```
SQL> UPDATE JOB
  2 SET STATUS = 'Complete'
  3 WHERE JOB_ID IN
  4 (
  5     SELECT JOB_ID
  6     FROM JOB_ASSIGNED
  7     WHERE EXTRACT(YEAR FROM STARTING_DATE) = 2000
  8 );

1 row updated.
```

9. Display job details of all those jobs where at least 25 workers are working.

```
SQL> SELECT J.*
  2 FROM JOB J
  3 WHERE J.JOB_ID IN
  4 (
  5     SELECT JOB_ID
  6     FROM JOB_ASSIGNED
  7     GROUP BY JOB_ID
  8     HAVING COUNT(WORKER_ID) >= 25
  9 );

no rows selected
```

10. Display all those jobs that are already completed.

```
SQL> SELECT *
  2 FROM JOB
  3 WHERE STATUS = 'Complete';
```

JOB_ID	TYPE_OF_JOB	STATUS
204	Welding	Complete

11. Find all the jobs, which will begin within the next two weeks.

```
SQL> SELECT *
  2 FROM JOB
  3 WHERE JOB_ID IN
  4 (
  5     SELECT JOB_ID
  6     FROM JOB_ASSIGNED
  7     WHERE STARTING_DATE BETWEEN SYSDATE AND SYSDATE + 14
  8 );
```

JOB_ID	TYPE_OF_JOB	STATUS
205	Painting	Running
206	Assembly	Pending

12. List all workers who have their wage per hour ten times greater than the wage of their managers.

```
SQL> SELECT W.WORKER_ID,
2         W.NAME,
3         W.WAGE_PER_HOUR,
4         M.NAME AS MANAGER_NAME,
5         M.WAGE_PER_HOUR AS MANAGER_WAGE
6 FROM WORKER W
7 JOIN WORKER M
8     ON W.MANAGER_ID = M.WORKER_ID
9 WHERE W.WAGE_PER_HOUR >= 10 * M.WAGE_PER_HOUR;
```

WORKER_ID	NAME	WAGE_PER_HOUR
103	David	1200
102	John	1000
104	Robert	10000

13. List the names of workers who have been assigned the job of molding.

```
SQL> SELECT W.NAME
2 FROM WORKER W
3 JOIN JOB_ASSIGNED JA
4     ON W.WORKER_ID = JA.WORKER_ID
5 JOIN JOB J
6     ON JA.JOB_ID = J.JOB_ID
7 WHERE LOWER(J.TYPE_OF_JOB) = 'molding';
```

NAME
Robert

14. What is the total number of days allocated for packaging the goods for all the workers

```
SQL> SELECT SUM(JA.NUMBER_OF_DAYS) AS TOTAL_DAYS
2 FROM JOB_ASSIGNED JA
3 JOIN JOB J
4     ON JA.JOB_ID = J.JOB_ID
5 WHERE LOWER(J.TYPE_OF_JOB) = 'packaging';
```

TOTAL_DAYS
30

together.

15. Which workers receive higher than average wage per hour.

SQL BOOK 2

```
SQL> SELECT *
2 FROM WORKER
3 WHERE WAGE_PER_HOUR >
4 (
5     SELECT AVG(WAGE_PER_HOUR)
6     FROM WORKER
7 );
```

WORKER_ID	NAME	WAGE_PER_HOUR
104	Robert	10000
102	John	1000

16. Display details of workers who are working on more than one job.

```
SQL> SELECT W.*
2 FROM WORKER W
3 WHERE W.WORKER_ID IN
4 (
5     SELECT WORKER_ID
6     FROM JOB_ASSIGNED
7     GROUP BY WORKER_ID
8     HAVING COUNT(JOB_ID) > 1
9 );
```

no rows selected

17. Which workers having specialization in polishing start their job in December?

```
SQL> SELECT W.*
2 FROM WORKER W
3 JOIN JOB_ASSIGNED JA
4 ON W.WORKER_ID = JA.WORKER_ID
5 WHERE LOWER(W.SPECIALISED_IN) = 'polishing'
6 AND EXTRACT(MONTH FROM JA.STARTING_DATE) = 12;
```

WORKER_ID	NAME	WAGE_PER_HOUR
102	John	1000
101	John	1000

18. Display details of workers who are specialized in the same field as that of Mr. Cacophonix or have a wage per hour more than any of the workers.

```

SQL> SELECT *
  2 FROM WORKER
  3 WHERE SPECIALISED_IN =
  4 (
  5     SELECT SPECIALISED_IN
  6     FROM WORKER
  7     WHERE NAME = 'Cacophonix'
  8 )
  9 OR WAGE_PER_HOUR > ANY
 10 (
 11     SELECT WAGE_PER_HOUR
 12     FROM WORKER
 13 );

```

WORKER_ID	NAME		WAGE_PER_HOUR
SPECIALISED_IN		MANAGER_ID	
101	Cacophonix		100
	Welding		
102	John		1000
	Polishing	101	
103	David		1200
	Welding	101	
104	Robert		10000
	Molding	102	
105	Smith		150
	Packaging	102	
106	Peter		200
	Polishing	103	

6 rows selected.

19. For each manager managing more than one worker, display the highest hourly wage paid to a worker, reporting to that manager.

```

SQL> SELECT M.WORKER_ID AS MANAGER_ID,
  2     M.NAME AS MANAGER_NAME,
  3     MAX(W.WAGE_PER_HOUR) AS HIGHEST_WORKER_WAGE
  4 FROM WORKER M
  5 JOIN WORKER W
  6     ON M.WORKER_ID = W.MANAGER_ID
  7 GROUP BY M.WORKER_ID, M.NAME
  8 HAVING COUNT(W.WORKER_ID) > 1;

```

MANAGER_ID	MANAGER_NAME	HIGHEST_WORKER_WAGE
102	John	10000
101	Cacophonix	1200

20. Display the specialization of the third lowest paid worker.

```
SQL> SELECT SPECIALISED_IN
 2 FROM
 3 (
 4     SELECT SPECIALISED_IN,
 5             DENSE_RANK() OVER (ORDER BY WAGE_PER_HOUR) AS RANK_NO
 6     FROM WORKER
 7 )
 8 WHERE RANK_NO = 3;

SPECIALISED_IN
-----
Polishing
```